TECHNICAL EXPLAINER, SIMPLE VERSION

How the AI Works, and Everything Around It

ResilienceAI | BGSW AI Hackathon | Status 10 October 2026

The most important fact first: we did not train the AI model. We use **Qwen3-4B**, a ready-made, free, open model published by Alibaba's Qwen team. We teach it our job by what we *send* it every time (instructions, the service map, measurements and past results), not by changing the model. The hackathon slides say no custom model training is required. Say this clearly: judges respect it, and it is how most real products use AI.

1. What an AI language model is

A language model is a program that is extremely good at one game: **guess the next word**. Give it "The cart service is ..." and it predicts which word most likely comes next, then the next, and so on. Doing this one word at a time, it can write whole answers.

Inside, it is a giant mathematical function with billions of adjustable numbers called **parameters** (think of them as tiny dials). Qwen3-4B has about **4 billion** of them. The values of those dials are the model's "knowledge".

2. How a model like Qwen3 gets trained (done by its makers, not by us)

Stage	In simple words	Analogy
1. Pre-training	The model reads an enormous amount of text and code (its publishers report trillions of words, in over 100 languages). For every sentence it guesses the next word, checks the real word, and nudges its dials slightly to guess better next time. Repeated billions of times on thousands of powerful chips for weeks.	A child reading every book in a library and learning how language works.
2. Instruction tuning	It is then shown many examples of a question and a good answer, so it learns to follow instructions and answer helpfully instead of just continuing text.	A tutor showing the right way to answer homework questions.
3. Feedback / reasoning training	Answers are scored (by people or by automatic checks such as "did the maths come out right?"), and the model is rewarded for better ones. This is also where it learns to **think step by step** before answering.	Practice tests with a teacher marking the work.

Why we did not train our own: training even a small model from scratch takes huge amounts of data, computing power and time. It is not needed here, because a good general model can already reason about services, errors and risks when we give it the right information.

3. How the model runs on our laptop

Piece	What it is	Simple explanation
Ollama	Free program that runs AI models locally	The "engine" that loads the model and answers our requests. No internet and no API key needed after download.
qwen3:4b	The model file (about 2.5 GB)	The 4 billion dials, stored compactly.

Piece	What it is	Simple explanation
Quantisation	Storing each dial with fewer bits (about 4 bits instead of 16)	Like saving a photo as a smaller JPEG: slightly less precise, 3-4x smaller, so it fits in a laptop.
GPU (RTX 4050, 6 GB)	The graphics card does the maths	Thousands of small calculators working in parallel. Much faster than the main processor.
Context window (8,192 tokens)	How much text the model can look at at once	Its short-term memory. Our request is about 3,000 tokens (a token is roughly a word or part of a word).

A real problem we solved: at first we gave the model a 16,000-token memory. That made it too big for the graphics card, so 23% spilled onto the slow main processor and planning took over 10 minutes. We measured our request (about 3,000 tokens), cut the memory to 8,192, the model fit 100% on the graphics card, and planning dropped to about 2.5 minutes.

4. How we teach it our job without training ("prompting")

Every time we ask for a plan, we send the model a package of information. It reads it fresh each time. This is called **in-context learning**: the knowledge is in the request, not in the model.

What we send	What it contains	Why
Instructions (system prompt)	Rules: rank experiments by user impact, prefer services many others depend on, do not repeat what is already known, give a specific prediction and a reason backed by evidence, stay within limits (5-30 s, 50-3000 ms delay).	Tells it what a good plan looks like.
Dependency map	Which of the 11 services calls which, which are critical, and the 3 user journeys (browse, cart, checkout).	So it knows what breaks what.
Healthy baseline	Normal numbers: 0% errors, p95 latency 26 ms.	So it knows what "normal" is.
Fault catalogue	The only 3 faults allowed: service outage, crash, network delay, with their limits.	So it only picks things we can actually do.
Past results	A summary of every experiment already run: target, errors, recovery, data loss.	Its **memory**: it avoids repeats and builds on findings.

Thinking mode: Qwen3 first writes out its reasoning ("redis-cart is stateful and used by cart and checkout... prior runs showed data loss...") and only then gives the answer. The dashboard shows this reasoning live.

Fixed answer format (structured output): we give the model a strict form with boxes: rank, target, failure type, duration, delay, prediction, reason. Ollama forces the model to fill exactly that form ("constrained decoding"), so we always get clean data a program can check, never free text.

Temperature 0.2: a setting that makes answers more consistent and less random.

5. The AI suggests; our code decides (safety)

The AI never runs commands. Every suggestion goes through a **safety gate** written in normal code:

- Is the target on the allowlist? Only 11 shop services in our own sandbox (namespace *boutique*); never the load generator, Kubernetes itself, or anything outside.
- Is the fault type one of the 3 allowed ones?
- Are the numbers within limits (duration, delay)?

- Is the shop healthy before we start? If not, refuse.

Proof it matters: in one run the small model asked for 300-second experiments, 10x over the limit. The gate rejected all of them and nothing ran. We then put the limits into the answer form itself, and the next plan was 100% valid. We also tested 8 deliberately dangerous requests (outside targets, a 10-minute fault, a shell command typed as a fault name): all refused.

6. The whole system, piece by piece

Piece	Technology	Job
The shop	Google Online Boutique: 11 services in Go, C#, Python, Node.js, Java	The app we test.
Practice data centre	Kubernetes in Docker (kind), on WSL2	Runs the shop's containers on the laptop, restarts crashed ones.
Fake shoppers (probe)	Python, 2-4 parallel users	Browse, add to cart, check out nonstop; time every request.
Telemetry	Python	Every 3 s: error rate, p95 latency, per-journey numbers; works out which service failed from the shop's own error messages.
Fault makers	kubectl and Linux traffic control (tc netem)	Outage = scale service to 0 copies. Crash = delete the container (repeatedly to hold it). Delay = add milliseconds to all traffic into a service.
Safety controller	Python	Validate, health check, inject, watch, kill switch, clean up, measure recovery, save evidence.
AI planner	Qwen3-4B via Ollama (Claude optional)	Ranks experiments with reasons.
Backend API	FastAPI (Python)	Connects the dashboard to everything; one job at a time.
Dashboard	HTML + JavaScript (AJAX)	Fetches small updates every 2 s and changes only the numbers and charts, so it never flickers.

7. What happens when you click "Let AI choose & inject errors"

#	Step	Time
1	The backend collects the map, baseline, fault list and past results.	instant
2	It sends them, with the instructions, to Qwen3 on the graphics card.	instant
3	Qwen3 thinks out loud (shown live on the dashboard), then fills in the ranked plan form.	~2.5 min
4	The safety gate checks each proposal: accepted or rejected with a reason.	instant
5	The controller watches the shop for 15 s to confirm it is healthy.	~18 s
6	It injects the AI's top pick (for example: delete the redis-cart container).	1 s
7	Telemetry shows the damage live; every request is recorded.	until you act
8	Kill switch: freeze as is. Rollback: restore the original state, then wait until real traffic is healthy and report the recovery time.	30-70 s
9	Everything is saved as a run record, which becomes "past results" for the AI's next plan.	instant

8. Words judges may use

Word	Meaning
LLM	Large language model: an AI that reads and writes text by predicting the next word.
Parameters	The model's adjustable numbers (4 billion in ours).
Inference	Using a trained model to get an answer (what we do). Training is building the model (what its makers did).
Fine-tuning	Extra training of an existing model on your own examples. We did not need it.
Prompt / system prompt	The instructions and information sent to the model.
In-context learning	The model uses information given in the request, without retraining.
Structured output	Forcing the answer into a fixed format (JSON) that code can check.
Quantisation	Compressing the model's numbers so it fits on small hardware.
Hallucination	When an AI states something false. We guard against it: the AI only proposes, and all measurements come from real traffic, never from the AI.
p95 latency	95% of requests were faster than this.
Blast radius	How much a fault is allowed to affect. Ours: one service at a time, sandbox only.

9. Likely technical questions, short answers

Question	Answer
Did you train the model?	No. We use the open Qwen3-4B model as-is and give it our context in each request. Training was not required, and this keeps it simple, cheap and explainable.
Why a local model?	Free, private, works offline, no API key. The code also supports Claude for stronger reasoning.
Why trust its choices?	We don't have to: every proposal is checked by code, and results are measured, not guessed.
Does it learn over time?	Yes, through memory, not training: every result is fed back into the next request.
Was the AI ever wrong?	Yes, usefully. It predicted a cart outage would only hurt the cart; the test showed it took down the whole site. Wrong predictions are exactly what chaos testing is for.